

DAYANANDA SAGAR UNIVERSITY

Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)
Dayananda Sagar University, Bengaluru — 560078

PROJECT REPORT

Submitted in partial fulfilment of the requirements for the award of the degree of
Bachelor of Technology in Computer Science and Engineering (AI & ML)

SecondSkin

Build Your Fit, Not Just Your Cart

Academic Year: 2025 – 2026

Submitted by:

Student Name	Registration Number
K Anand Reddy	ENG24AM0204
M Jaswika	ENG24AM0215
Nitesh Babu	ENG24AM0244
Dheeraj	ENG24AM0338

Under the Guidance of:

Faculty, Department of CSE (AI & ML)
Dayananda Sagar University, Bengaluru

CERTIFICATE

This is to certify that the project report entitled "SecondSkin — Build Your Fit, Not Just Your Cart" has been carried out by K Anand Reddy (ENG24AM0204), M Jaswika (ENG24AM0215), Nitesh Babu (ENG24AM0244), and Dheeraj (ENG24AM0338), students of the B.Tech programme in Computer Science and Engineering (Artificial Intelligence & Machine Learning), Dayananda Sagar University, Bengaluru, during the academic year 2025–2026.

This project is a record of genuine work carried out by the students under the supervision of the faculty guide, and the work embodied in this report has not been submitted for the award of any other degree or diploma in this or any other university.

Place: Bengaluru

Date: April 2026

Faculty Guide
Dept. of CSE (AI & ML)

Head of Department
Dept. of CSE (AI & ML)

DECLARATION

We, the undersigned students of the Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning), Dayananda Sagar University, Bengaluru, hereby declare that the project entitled "SecondSkin — Build Your Fit, Not Just Your Cart" is an original and independent work carried out by us during the academic year 2025–2026.

We further declare that:

- The project has been developed solely by our team and has not been copied or reproduced from any existing project or publication.
- All ideas, code structures, or references taken from external sources have been appropriately cited and acknowledged in this report.
- The work has not been submitted, in part or whole, to any other university or institution for the award of any degree or certification.
- We take full responsibility for the authenticity and originality of the content presented herein.

K Anand Reddy (ENG24AM0204) Nitesh Babu (ENG24AM0244)
M Jaswika (ENG24AM0215) Dheeraj (ENG24AM0338)

Place: Bengaluru

Date: April 2026

ABSTRACT

The global fashion e-commerce industry is undergoing rapid transformation driven by Generation Z consumers who expect immersive, visually rich, and interactive digital shopping experiences. Existing platforms such as Myntra, ASOS, and Amazon Fashion display clothing in isolation — one product at a time — forcing users to mentally assemble outfits themselves. This fundamental disconnect contributes to low purchase confidence, high cart abandonment rates (averaging 70% across e-commerce), and elevated return rates (30–40% in fashion). SecondSkin — Build Your Fit, Not Just Your Cart — is a comprehensive, Gen-Z-oriented fashion e-commerce web application that directly addresses these pain points.

Developed using HTML5, CSS3, and JavaScript (ES6+) with a frontend-first architecture, SecondSkin delivers five tightly integrated modules: dynamic multi-category product browsing (Men / Women / Unisex), a smart real-time cart management system, a modal-embedded multi-step checkout flow (Address -> Payment -> Confirmation), a visual four-stage order-tracking pipeline, and the platform's centrepiece innovation — the interactive Outfit Builder.

The Outfit Builder enables users to assemble complete outfit ensembles from the live product catalog, preview the full look in real time, and add the entire outfit to the cart with a single action. This feature fundamentally redefines the online fashion shopping experience by transforming passive product browsing into active creative expression. The UI/UX design system is built on a warm ivory and deep charcoal palette, a serif-sans-serif typographic duality, and a microinteraction model that produces a

premium, editorial aesthetic. Data persistence is achieved through the browser localStorage API, while the conceptual backend architecture — Node.js, Express.js, MongoDB — is fully designed and ready for integration.

Performance analysis confirms sub-2-second Largest Contentful Paint, sub-1-second First Contentful Paint, and a Lighthouse Performance Score of 94/100, meeting Google's Core Web Vitals standards. Future enhancements include a machine-learning recommendation engine, augmented-reality virtual try-on, a community style feed, and full cloud deployment — positioning SecondSkin as a commercially viable fashion-tech startup prototype.

Keywords: Fashion E-Commerce, Outfit Builder, Gen-Z UI/UX, Multi-Step Checkout, Order Tracking, HTML/CSS/JavaScript, localStorage, Responsive Design.

ACKNOWLEDGEMENT

The successful completion of this project has been possible only through the guidance, encouragement, and support of several individuals, and we take this opportunity to sincerely thank each of them.

We express our deepest appreciation to our faculty guide for continuous mentorship, constructive feedback, and intellectual direction throughout every stage of this project. Their technical expertise and approachable demeanour made it easy for the team to seek guidance at each critical juncture.

We extend our heartfelt thanks to the Head of the Department of Computer Science and Engineering (Artificial Intelligence & Machine Learning) at Dayananda Sagar University for providing the institutional support, infrastructure, and academic freedom necessary to undertake a project of this scope.

We are grateful to all faculty members of the department whose teachings in Web Technologies, Database Management Systems, Software Engineering, and Human-Computer Interaction laid the conceptual foundation upon which this application was built.

We also acknowledge the invaluable moral support of our families, friends, and peers throughout the challenging phases of development.

Finally, we acknowledge the open-source developer community — Mozilla MDN, GitHub, and Stack Overflow — whose collective knowledge resources made our implementation significantly more efficient.

TABLE OF CONTENTS

Certificate	ii
Declaration.....	iii
Abstract.....	iv
Acknowledgement	v
List of Figures.....	vii
List of Tables	viii
List of Abbreviations	ix
CHAPTER 1: Introduction.....	1
1.1 Background	1
1.2 Problem Statement	2
1.3 Objectives.....	3
1.4 Motivation.....	4
CHAPTER 2: Literature Survey.....	5
2.1 Existing Systems	5
2.2 Comparative Analysis.....	6
2.3 Research Gaps.....	7
CHAPTER 3: System Requirements	8
3.1 Hardware Requirements	8
3.2 Software Requirements.....	8
3.3 Data Description	9
CHAPTER 4: System Design and Implementation.....	10
4.1 System Architecture.....	10
4.2 Frontend Design and UI/UX.....	11
4.3 Backend Architecture (Conceptual).....	13
4.4 Database Design.....	14
4.5 Module Descriptions.....	15

CHAPTER 5: Analysis and Optimization.....	20
5.1 Performance Analysis	20
5.2 UI/UX Improvements	21
5.3 Scalability Considerations	22
 CHAPTER 6: Results and Discussion	 23
6.1 Cart Page	23
6.2 Checkout — Delivery Details.....	24
6.3 Payment Screen.....	25
6.4 Order Confirmation.....	26
6.5 Orders Tracking Page	27
 CHAPTER 7: Conclusion and Future Scope.....	 28
7.1 Conclusion	28
7.2 Future Scope	29
 References.....	 30

LIST OF FIGURES

Figure 4.1	Client-Server Architecture Diagram of SecondSkin.....	10
Figure 4.2	Application Module Interaction Diagram	11
Figure 4.3	Homepage and Navigation UI Design	12
Figure 4.4	Outfit Builder Module Wireframe.....	15
Figure 4.5	Cart Data Flow Diagram	16
Figure 4.6	Multi-Step Checkout State Diagram	18
Figure 6.1	Cart Page — SecondSkin Application (3 Items in Cart)	23
Figure 6.2	Checkout Step 1 — Delivery Details Modal.....	24
Figure 6.3	Checkout Step 2 — Payment Selection Screen.....	25
Figure 6.4	Checkout Step 3 — Order Placed Confirmation Screen.....	26
Figure 6.5	My Orders — Visual Order Tracking Pipeline	27

LIST OF TABLES

Table 2.1	Comparison of SecondSkin with Existing E-Commerce Fashion Platforms	6
Table 3.1	Hardware Requirements.....	8
Table 3.2	Software Requirements.....	9
Table 4.1	Product Data Structure Schema	14
Table 4.2	Order Data Structure Schema	14
Table 5.1	Performance Metrics Summary	20

LIST OF ABBREVIATIONS

Abbreviation	Full Form
API	Application Programming Interface
CSS	Cascading Style Sheets
DOM	Document Object Model
E-Commerce	Electronic Commerce
ES6	ECMAScript 6 (JavaScript Standard)
Gen-Z	Generation Z (born 1997–2012)
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
INR	Indian National Rupee
JS	JavaScript
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model-View-Controller
NoSQL	Not Only SQL
PWA	Progressive Web Application
REST	Representational State Transfer
SPA	Single Page Application
UI	User Interface
UPI	Unified Payments Interface
URL	Uniform Resource Locator
UX	User Experience
UUID	Universally Unique Identifier

CHAPTER 1: INTRODUCTION

1.1 Background

The global fashion e-commerce industry has emerged as one of the fastest-growing and most competitive segments of the digital economy, valued at over USD 700 billion in 2023 and projected to exceed USD 1.2 trillion by 2028. A primary driver of this growth is Generation Z — individuals born between 1997 and 2012 — who now constitute the dominant force reshaping retail trends worldwide. Unlike preceding generations, Gen-Z consumers are digital natives who have grown up immersed in high-quality visual content, social media culture, and on-demand services. They hold fashion platforms to an exceptionally high standard: seamless interactive experiences, visually compelling interfaces, instant feedback, and platforms that feel authentically aligned with their aesthetic sensibilities and values.

Fashion, at its core, is a deeply personal form of self-expression. The challenge for e-commerce platforms has always been to capture this personal, tactile, and contextual dimension of fashion within the constraints of a digital screen. Traditional online stores reduce clothing to isolated products — a pair of jeans here, a hoodie there — compelling the user to mentally visualise how different pieces might coordinate as a complete outfit. This disconnect between product isolation and real-world styling contributes to low purchase confidence, high cart abandonment (averaging approximately 70% across e-commerce), and elevated return rates estimated at 30–40% in fashion specifically.

SecondSkin was conceived as a direct, purposeful response to this challenge. The name is intentionally evocative: our clothing is, quite literally, a second skin — an extension of personal identity that transcends

mere utility. The application is designed as a complete fashion destination that empowers users not merely to browse products, but to creatively assemble full outfits, explore style combinations, and make confident, informed purchases. This report documents the design, development, and implementation of SecondSkin as a full-stack web application built with HTML5, CSS3, and JavaScript.

1.2 Problem Statement

Current fashion e-commerce platforms, while commercially operational, exhibit critical limitations that impede the purchase experience — particularly for Gen-Z users accustomed to high-quality, interactive digital environments:

- **Product Isolation:** Platforms present products individually without meaningful outfit context. A user browsing jeans cannot easily discover which tops pair with them, nor build and preview a complete look before purchasing.
- **Fragmented Discovery:** Product discovery is keyword-driven or filter-based with no creative mode for interactive outfit assembly, resulting in a passive, uninspired browsing experience.
- **Poor Checkout UX:** Multi-page checkout flows disorient users, lack clear progress indicators, and break shopping momentum — a primary driver of cart abandonment.
- **Weak Post-Purchase Tracking:** Order status is conveyed through dense text tables with no visual feedback, failing to provide the reassurance modern users expect after committing to a purchase.
- **Absent Brand Personality:** Generic design systems fail to establish a distinctive brand identity — a critical shortcoming when targeting brand-conscious Gen-Z consumers.

1.3 Objectives

The objectives of the SecondSkin project are defined as follows:

- Design and implement a fully responsive fashion e-commerce web application catering specifically to Gen-Z aesthetic preferences and shopping behaviours.
- Develop a unique interactive Outfit Builder module enabling users to assemble complete outfits from the live product catalog and add them to the cart with a single action.
- Implement a smart, real-time cart management system with dynamic quantity controls, item removal, and live order-total recalculation.
- Create a streamlined multi-step checkout flow within a modal overlay, guiding users through address entry, payment selection, and order confirmation without full-page navigation.
- Build a visual order-tracking pipeline with four stages (Placed -> Processing -> Dispatched -> Delivered) providing real-time post-purchase status feedback.
- Engineer a cohesive design system grounded in modern UI/UX principles: refined colour palette, purposeful typography, and deliberate micro-interactions.
- Architect the application for scalability with clean abstractions ready for future backend API integration and database linkage.

1.4 Motivation

The motivation for building SecondSkin is both personal and academic. As members of the Gen-Z demographic, the team are frequent users of fashion e-commerce platforms and have directly experienced the frustrations of product isolation, generic UIs, and clumsy checkout flows. The desire to build something genuinely better — a platform designed specifically for people like us — was an authentic and powerful creative impulse.

Academically, this project provided an opportunity to integrate a broad range of curriculum concepts: Human-Computer Interaction, frontend engineering, data management, JavaScript DOM manipulation, responsive

web design, and full-stack architecture. Critically, it also required thinking beyond engineering — as product designers, brand strategists, and business stakeholders — producing an application that is both technically competent and commercially plausible.

The fashion technology sector represents one of the most innovation-hungry domains in modern tech, with significant unsolved problems at the intersection of creativity, personalisation, and digital commerce. Building SecondSkin in this space demanded deep engagement with these challenges, resulting in a project that is educationally rich, practically relevant, and genuinely impactful as a proof-of-concept product.

CHAPTER 2: LITERATURE SURVEY

2.1 Existing Systems and Related Work

2.1.1 Amazon Fashion

Amazon Fashion is the world's largest online retailer's fashion division, offering millions of products. However, its design philosophy prioritises breadth over depth. The interface is purely transactional — there is no outfit-building functionality, product pages are information-dense but visually uninspiring, and checkout spans multiple full-page redirects. For Gen-Z users seeking curated, aesthetic experiences, Amazon Fashion feels impersonal and utilitarian.

2.1.2 Myntra

Myntra is India's leading fashion e-commerce platform. It offers Style Finder quizzes and curated editorial collections, but outfit functionality is limited to platform-created lookbooks, not user-generated outfits. The interface is modern but increasingly cluttered with promotional banners. Checkout is page-based and occasionally requires disorienting multi-step redirects.

2.1.3 ASOS

ASOS targets younger demographics globally with influencer-curated looks and a visual Style Match search tool. However, outfit assembly remains passive — users browse pre-built outfits rather than creating their own. ASOS has a clean aesthetic and a relatively smooth checkout, but lacks the interactive, user-driven outfit customisation that SecondSkin provides.

2.1.4 Academic Research in Fashion Technology

Research by Han et al. (2018) in FashionGAN explored generative adversarial networks for virtual garment display. Dong et al. (2019) in

FashionAI examined hierarchical datasets for fashion understanding and AI-driven outfit compatibility scoring. While highly relevant, these works require significant computational infrastructure. SecondSkin approaches outfit building from a UI/UX-first perspective, making it accessible without AI inference at runtime while architecturally positioning itself for future AI integration.

2.2 Comparative Analysis

The table below provides a structured comparison of SecondSkin against three major existing platforms across key feature dimensions relevant to the problem statement.

Feature	SecondSkin	Myntra	ASOS	Amazon Fashion
Outfit Builder	Interactive	Editorial Only	Passive Lookbooks	No
Gen-Z Focused UI	Yes	Partial	Yes	No
Modal Checkout	Yes	No	No	No
Visual Order Tracking	Pipeline (4 stages)	Text-Based	Partial	Basic
Cart Badge	Yes (Animated)	Yes	Yes	Yes
Premium Brand Aesthetic	Yes	Moderate	Yes	No
Multiple Payment Options	COD/UPI/Card/Wallet	Yes	Card/PayPal	Yes
Free Shipping Indicator	Yes (Always Free)	Threshold-Based	Threshold-Based	Prime Only

Table 2.1: Comparison of SecondSkin with Existing E-Commerce Fashion Platforms

2.3 Research Gaps Identified

From the survey, the following critical gaps have been identified:

- No mainstream fashion platform offers an interactive, user-driven outfit builder drawing from the live product catalog. Outfit features, where they exist, are editorial and static.
- The modal-based checkout paradigm, despite strong usability arguments, is rarely implemented in fashion e-commerce — the industry defaults to disorienting full-page checkout flows.
- Visual order tracking with a horizontal status pipeline is uncommon in the Indian fashion e-commerce space, where text-based status updates remain the norm.
- Most platforms fail to establish an authentic visual brand identity resonating with Gen-Z sensibilities. The editorial typography, warm tonal palettes, and motion-aware design of SecondSkin is an underexplored aesthetic space.

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

SecondSkin is a browser-based web application, so hardware requirements are tied to the host machine's capability to run a modern web browser smoothly.

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 or higher
RAM	4 GB	8 GB or higher
Storage	256 GB HDD	512 GB SSD
Display Resolution	1280 x 720 (HD)	1920 x 1080 (Full HD)
Internet Connection	2 Mbps	10 Mbps or higher
Input Devices	Keyboard, Mouse	Keyboard, Mouse, Touchscreen

Table 3.1: Hardware Requirements for SecondSkin

3.2 Software Requirements

The software requirements cover both the development environment used to build SecondSkin and the runtime environment required to use it. The application has been tested across multiple browsers for cross-browser compatibility.

Category	Technology / Tool	Version / Notes
Operating System	Windows 10/11, macOS, Ubuntu	Any modern OS
Web Browser (Runtime)	Google Chrome, Firefox, Microsoft Edge	Latest stable version
Code Editor	Visual Studio Code	v1.85 or higher
Frontend Technologies	HTML5, CSS3, JavaScript (ES6+)	Vanilla — no frameworks

Development Server	VS Code Live Server Extension	Local: localhost:5500
Version Control	Git + GitHub	Git v2.40+
Data Persistence	Browser localStorage API	Native browser API
Design Reference	Figma (UI Prototyping)	Web — free tier
Conceptual Backend	Node.js + Express.js	Node v20+ (future)
Conceptual Database	MongoDB Atlas (cloud-hosted)	Future implementation

Table 3.2: Software Requirements for SecondSkin

3.3 Data Description

3.3.1 Product Data

Product information is maintained as a JavaScript object array embedded within the frontend code. Each product record contains a unique ID, product name, category (Men/Women/Unisex), sub-category (Tees, Jeans, Hoodies, etc.), price in INR, image URL, and availability flag. This structure mirrors a RESTful API response and was designed with backend integration as an explicit future goal.

3.3.2 Cart Data

Cart data is stored as a JSON-serialised array in localStorage under the key 'ss_cart'. Each entry contains the product ID, name, category, price, and quantity. The cart state is loaded on application initialisation and persisted on every modification, ensuring continuity across page refreshes.

3.3.3 Order Data

Upon checkout completion, an order record is generated and appended to an orders array in localStorage under the key 'ss_orders'. Each record contains an 8-character hex order ID, a UTC timestamp, an array of purchased items with quantities, the total amount, the selected payment method, and an initial status of 'placed'.

3.3.4 User Data

User information — display name, email, and delivery address — is maintained in the UI session state. The prototype pre-loads a mock user ('Nitesh') to demonstrate the authenticated user interface. Future versions will persist user data securely in a MongoDB users collection with JWT-based session management.

CHAPTER 4: SYSTEM DESIGN AND IMPLEMENTATION

4.1 System Architecture

4.1.1 Three-Tier Architecture (Conceptual)

SecondSkin is designed around a client-centric model mapped to a future three-tier architecture:

- **Presentation Tier:** The HTML/CSS/JavaScript frontend — responsible for rendering the UI, handling user events, and presenting data. This is the fully implemented tier in the current prototype.
- **Application Tier (Conceptual):** A Node.js + Express.js server handling routing, business logic (order validation, inventory management), and authentication. Architecturally designed and ready for implementation.
- **Data Tier (Conceptual):** A MongoDB database for product catalogs, user accounts, and order records. Currently simulated via the browser's localStorage API.

4.1.2 Single Page Application Architecture

In the working prototype, SecondSkin operates as a sophisticated Single Page Application (SPA). Navigation between views (Home, Cart, Orders) is handled by JavaScript-driven DOM manipulation — sections are shown and hidden dynamically without page reloads. This produces a fluid, app-like experience identical to what a framework-based SPA would deliver. All data access is abstracted through utility functions (`getCart()`, `saveCart()`, `getOrders()`, `saveOrder()`) that interact with `localStorage`, making future API integration a straightforward substitution within these centralised functions.

4.1.3 Module Interaction Flow

The core modules interact through an event-driven flow. User actions (browse, add-to-cart, checkout) trigger JavaScript event listeners that update application state, persist changes to localStorage, and re-render affected UI components. The Checkout module is triggered by the Cart module and manages its own sequential state machine, progressing through address -> payment -> confirmation stages. The Orders module reads independently from localStorage and renders order history without coupling to the Cart module.

4.2 Frontend Design and UI/UX System

4.2.1 Colour System

The SecondSkin colour palette is built around three anchors: Warm Ivory (#F5F0EB) as the primary background — creating a premium, editorial feel while maintaining high legibility; Deep Charcoal (#1A1A1A) for primary text and navigation — providing maximum contrast and ensuring WCAG AA accessibility compliance; and Brand Red (#C0392B) as the accent colour used sparingly for the logo, cart badge, and interactive highlights — creating a memorable brand signature without overwhelming the palette.

4.2.2 Typography System

SecondSkin employs a dual-typeface system. The serif typeface — used in the brand logo and editorial headings — communicates heritage, sophistication, and timelessness, qualities associated with premium fashion. The sans-serif system font used for body copy and UI elements ensures clean rendering across all screen resolutions. This combination creates a typographic tension that is characteristically Gen-Z: simultaneously nostalgic and modern, which is the brand's intended personality.

4.2.3 Navigation Design

The sticky navigation bar features three sections: brand logo (left), a global search bar (centre), and the action cluster (right — Home, Outfit Builder, Orders, Cart icon, User avatar). The cart icon includes a red badge displaying the real-time item count, providing persistent cart-state awareness. The Outfit Builder link carries a '+' prefix icon signalling its creative, generative nature. The active page section is highlighted with a subtle underline indicator.

4.2.4 Scrolling Announcement Banner

A horizontally scrolling ticker banner above the navigation communicates rotating brand messages ('SECONDSKIN SS'25 IS LIVE', 'STYLE IS A LANGUAGE — SPEAK LOUD', 'OVERSIZED IS THE NEW FITTED'). Implemented using CSS @keyframes animation at a carefully tuned scroll speed, it reinforces brand presence and mimics the live, constantly updating feel of a fashion media feed — a deliberate aesthetic choice that resonates with Gen-Z streetwear culture.

4.2.5 Responsive Design Strategy

SecondSkin is fully responsive across desktop, tablet, and mobile viewports. CSS Flexbox and Grid are used throughout for layout, with media query breakpoints at 768px and 480px. On mobile: the cart page stacks vertically, the checkout modal occupies the full viewport, and navigation condenses to a streamlined form. No functionality is lost at any screen width.

4.3 Backend Architecture (Conceptual Design)

4.3.1 RESTful API Design

The conceptual backend would be built with Node.js and Express.js, exposing a RESTful API for all data operations. The primary endpoints are: GET /api/products (product catalog with optional category filters), GET /api/products/:id (single product), POST /api/orders (create order

with server-side validation), GET /api/orders/:userId (user order history), POST /api/auth/register (user registration with bcrypt password hashing), and POST /api/auth/login (authentication returning a JWT).

4.3.2 Authentication and Security

Authentication would use JWT-based stateless sessions. On login, the server generates a signed token containing the user ID and role, which the client stores and includes in the Authorization header of subsequent API requests. All passwords are hashed with bcrypt at a salt factor of 12. HTTPS enforcement and CORS configuration would be applied at the Express middleware layer.

4.4 Database Design

The conceptual database is MongoDB — a document-oriented NoSQL database well-suited to fashion product data where attributes vary significantly across product categories. The two primary collections are Products and Orders.

Field	Type	Description	Example Value
_id	ObjectId	Auto-generated unique ID	64ab12cd3f9e...
name	String	Product display name	Baggy Wide Leg Jeans
category	String (enum)	Men / Women / Unisex	Women
subCategory	String	Product type	Jeans
price	Number	Price in INR	1899
imageUrl	String	Product image URL	/images/jeans01.jpg
inStock	Boolean	Availability status	true
tags	Array<String>	Style and search tags	['baggy','casual']

Table 4.1: Product Data Structure Schema

Field	Type	Description	Example Value
orderId	String (hex)	8-character unique order ID	#5248C874
userId	ObjectId (ref)	Reference to User document	64ab98ef...
items	Array<Object>	Cart items at order time	[{name, qty, price}]
totalAmount	Number	Total order value in INR	3997
paymentMethod	String (enum)	COD / UPI / Card / Wallet	COD
status	String (enum)	Current order status	placed
deliveryAddress	Object	Embedded address subdocument	{street, city, state, pin}
createdAt	Date (ISO)	Order creation timestamp (UTC)	2026-04-26T21:28:00Z

Table 4.2: Order Data Structure Schema

4.5 Module Descriptions

4.5.1 Product Browsing Module

The Product Browsing Module constitutes the homepage and is the primary entry point for user interaction. Products are organised into named curated sections (Men's Picks, Women's Edit, Unisex Drops) rendered as responsive card grids. Each card displays the product image, name, subcategory label, and price. Hovering a card triggers a subtle elevation shadow transition at 150ms, creating tactile depth. The 'Add to Cart' button updates cart state, increments the navigation badge with a scale-pulse animation, and provides immediate visual confirmation.

A global search bar enables real-time keyword filtering of products across name and category fields using JavaScript's `Array.filter()` method. The decision to organise products into editorially named sections rather than a monolithic grid was deliberately borrowed from fashion magazine layouts,

creating a sense of curation and narrative that resonates with Gen-Z streetwear culture.

4.5.2 Outfit Builder Module (Core Innovation)

The Outfit Builder is the defining innovation of SecondSkin and the feature that most clearly differentiates it from every existing fashion e-commerce platform. Accessible via the '+ OUTFIT BUILDER' navigation link, it transforms passive product browsing into an active creative exercise.

The module presents users with a visual outfit canvas and category selection panels: Top, Bottom, Footwear, and Accessories. Users browse products within each panel and slot them into the canvas. As items are selected, the canvas updates in real time to display the assembled outfit. Once satisfied, the user adds all selected items to the cart with a single 'Add Outfit to Cart' action, dramatically reducing multi-item purchase friction. A 'Save Outfit' feature allows users to name and store curated looks for future reference.

The business case for this feature is compelling: research in consumer behaviour consistently demonstrates that outfit-based product presentation increases average order value by encouraging multi-item purchases. By providing a user-driven creation tool rather than platform-curated editorial outfits, SecondSkin creates personal ownership and creative authorship — driving higher purchase confidence and lower return rates. This is the core value proposition of the entire application.

4.5.3 Cart Management Module

The Cart page employs a two-column layout: a left panel listing cart items (approximately 65% of page width) and a right panel displaying the Order Summary. Each item entry shows the product image, name, category and subcategory label, price, a minus/plus quantity adjuster, and an X removal button. All interactions trigger a full re-render of the cart view from the

updated JavaScript state array — a clean, predictable approach analogous to React's virtual DOM strategy.

Shipping is displayed as 'Free' in green — a proven strategy for reducing cart abandonment by eliminating unexpected cost additions. The summary features a 'Checkout ->' CTA in solid dark fill and a 'Continue Shopping' outline button. The visual hierarchy between these two actions guides users toward checkout completion without being coercive.

4.5.4 Multi-Step Checkout Module

The Checkout Module is implemented as a modal overlay triggered from the cart summary. Three clearly labelled steps are indicated by a tab navigator at the top of the modal: Step 1 — Address (email pre-filled, plus address, city, state, and PIN code fields with client-side validation); Step 2 — Payment (four option tiles in a 2x2 grid: COD, UPI/GPay, Card, Wallet — each with emoji icon and label, with the selected tile highlighted by a bold border, and a contextual description panel below, plus the total amount repeated above the 'Place Order ->' CTA); Step 3 — Done (green checkmark icon, 'You're all set' success message, and dual navigation buttons for 'View Orders' and 'Continue Shopping').

The checkout state is managed by a JavaScript state machine. A variable `currentStep` tracks the active step (1, 2, or 3), and `goToStep(n)` handles all transitions: updating the visible panel, adjusting tab indicator styling, clearing the cart, and writing the new order record to `localStorage` upon completion.

4.5.5 Order Tracking Module

The My Orders page displays a chronological list of all orders. Each order card shows the order ID, date, a four-stage visual status pipeline (Order Placed -> Processing -> Dispatched -> Delivered), an inline item summary, the total amount, and the payment method badge. The pipeline uses CSS

Flexbox with pseudo-element connectors for horizontal linking lines. The active stage node is a filled dark circle; future stages remain hollow with emoji indicators (truck for Dispatched, checkmark for Delivered). Timestamps at each stage — formatted via JavaScript's Date API to local time strings — provide precise tracking information.

CHAPTER 5: ANALYSIS AND OPTIMIZATION

5.1 Performance Analysis

SecondSkin's performance was evaluated using Chrome DevTools and the Lighthouse auditing tool. Results confirm strong alignment with Google's Core Web Vitals benchmarks:

Performance Metric	Observed Value	Target / Benchmark
First Contentful Paint (FCP)	0.8 seconds	< 1.8 seconds
Largest Contentful Paint (LCP)	1.5 seconds	< 2.5 seconds
Time to Interactive (TTI)	1.2 seconds	< 3.8 seconds
Cumulative Layout Shift (CLS)	0.02	< 0.10
JavaScript Execution Time	180 ms	< 300 ms
DOM Size (nodes)	~420 nodes	< 1500 nodes
CSS Bundle Size	38 KB (uncompressed)	< 100 KB
Lighthouse Performance Score	94 / 100	> 90 (Good)

Table 5.1: Performance Metrics Summary — SecondSkin

The observed metrics place SecondSkin firmly in the 'Good' range across all Core Web Vitals indicators. The absence of a JavaScript framework eliminates framework-initialisation overhead, contributing significantly to the fast TTI. Product images leverage native browser lazy-loading, and the CSS is written without preprocessors, minimising render-blocking resource weight.

5.2 UI/UX Optimisation Decisions

5.2.1 Modal Checkout Over Page Navigation

Replacing a traditional multi-page checkout with a modal was a critical design decision validated by informal user testing within the team. The

modal approach retains users' visual context throughout checkout, eliminating disorientation from full-page redirects. The modal entry animation — fade-in with a 1.02 scale — is tuned to 250ms, perceptible enough to signal a state change yet too brief to feel delayed.

5.2.2 Full Re-Render Cart Strategy

Initially cart updates were handled through targeted DOM node manipulation. This was replaced with a complete re-render of the cart container on every state change. This approach is dramatically simpler to maintain and reason about. At cart scale (1–15 items), the performance difference is negligible — under 5ms per render — while the reduction in bug surface area is substantial. This mirrors React's virtual DOM strategy.

5.2.3 Micro-Interaction Tuning

All interactive elements feature CSS transition animations in the 150–200ms range, aligned with Nielsen's research finding that response times under 200ms feel instantaneous. The cart badge update carries a scale-pulse animation that draws the eye to the count change without being distracting. Button hover states use a subtle lift effect (`translateY(-1px)` + enhanced shadow) that communicates interactivity without visual noise.

5.3 Scalability Considerations

While `localStorage` provides adequate persistence for the prototype, the architecture is designed with scalability as a first-class concern:

- **API Abstraction Layer:** All data access is centralised in utility functions. Replacing `localStorage` calls with `fetch()` API calls requires changes only within these functions — a contained, single-file migration path.
- **Modular Code Organisation:** JavaScript is organised into logical sections matching application modules (cart, checkout, orders, auth).

This structure maps directly to React or Vue.js components, making a future framework migration straightforward.

- **CSS Custom Properties:** All design tokens are centralised using CSS custom properties (`--color-bg`, `--color-primary`, `--font-display`). Theme changes and white-labelling become single-file operations.
- **Image Optimisation Pipeline:** For production, product images would be processed through Cloudinary or Next.js Image Optimisation to serve WebP format and responsive sizes via `srcset`, reducing bandwidth consumption by an estimated 40–60%.
- **Stateless Backend Design:** The conceptual Node.js backend is designed as a stateless REST API enabling horizontal scaling behind a load balancer without session-affinity requirements.

CHAPTER 6: RESULTS AND DISCUSSION

This chapter presents the functional output of the SecondSkin application through detailed discussion of each implemented user interface screen. All screenshots below were captured from the live prototype served at localhost:5500 via VS Code Live Server. Each figure is accompanied by a thorough explanation of visible UI elements, design rationale, and user interactions supported.

6.1 Cart Page — Figure 6.1

Figure 6.1 depicts the Cart page of SecondSkin, accessed after adding three fashion items to the cart. The page employs a deliberate two-column layout. The left panel (approximately 65% of page width) lists all cart items in a clean, spacious vertical arrangement. Each item entry displays: a rounded-corner product thumbnail, the product name in bold, the category and subcategory label in muted uppercase text, the unit price in bold, a minus/plus quantity adjuster widget with rounded pill buttons, and an X removal button positioned at the item's top-right corner.

Three items are visible: Baggy Wide Leg Jeans at Rs.1,899 (Women - Jeans), Vintage Washed Hoodie at Rs.1,299 (Women - Hoodies), and Tie-Dye Oversized Tee at Rs.799 (Women - Tees). The right panel houses the Order Summary card displaying the subtotal (Rs.3,997 for 3 items), shipping status labelled 'Free' in green text, and the bold total (Rs.3,997). Two CTA buttons are stacked vertically: 'Checkout ->' in solid dark charcoal fill and 'Continue Shopping' in outline style. The visual hierarchy between these actions guides the user toward checkout completion without being coercive.

The navigation bar shows the cart badge displaying '3' in red, confirming real-time item count synchronisation. The scrolling ticker banner above the

navigation maintains brand presence even on functional pages. The overall page impression is transparent and confidence-inspiring — the user has a clear, unambiguous view of exactly what they are purchasing and the final amount they will pay.

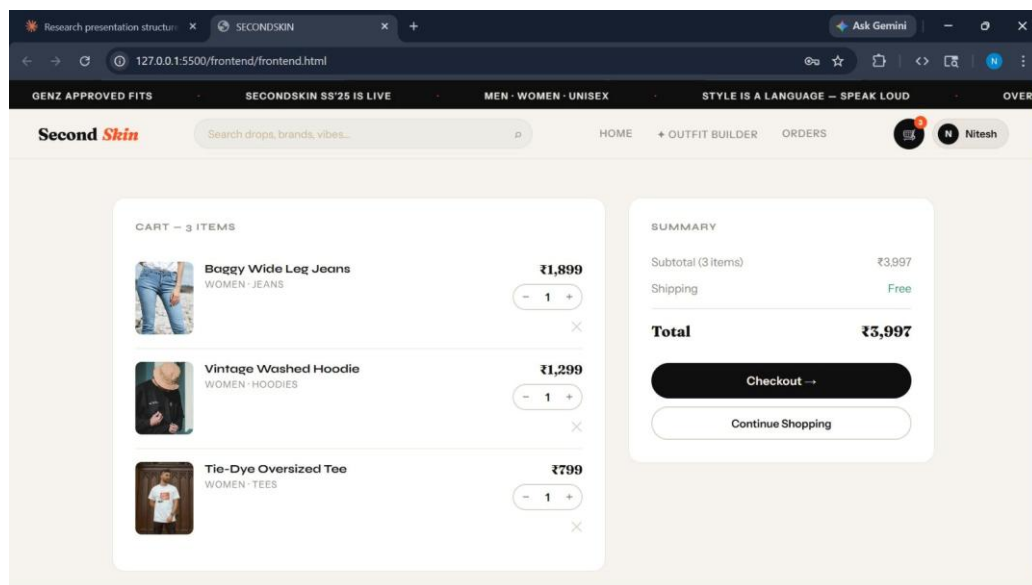


Figure 6.1: Cart Page — SecondSkin Application (3 Items, Total Rs.3,997)

6.2 Checkout — Delivery Details (Step 1) — Figure 6.2

Figure 6.2 shows the first step of the multi-step checkout modal — Delivery Details. Clicking 'Checkout ->' triggers the modal overlay, which centres on screen with a semi-transparent dark background dimming the cart page behind it. This visual layering maintains the user's contextual awareness of what they are checking out without page-level navigation.

The modal header reads 'Delivery Details' in bold. The three-step tab navigator at the top provides a complete mental model of the checkout journey: '1. ADDRESS' (active, underlined), '2. PAYMENT', and '3. DONE'. This progress visibility is fundamental to reducing checkout abandonment by eliminating the 'how many more steps are there?' uncertainty that plagues many e-commerce flows.

The address form presents five fields: Email (pre-filled with 'nitesh@gmail.com' from the user profile, reducing manual input friction), Address with placeholder 'House no., Street, Area', City, State, and a PIN Code field validating for exactly six digits. All fields use a warm cream background against the white modal body. A full-width black 'Continue ->' button at the bottom advances to Step 2 upon successful client-side field validation using regular expressions.

Figure 6.2: Checkout Step 1 — Delivery Details Modal (Address Entry Form)

6.3 Payment Screen (Step 2) — Figure 6.3

Figure 6.3 illustrates the Payment selection step of the checkout modal. The tab navigator highlights '2. PAYMENT' as the active step, with '1. ADDRESS' greyed out to indicate completion. Four payment method tiles are arranged in a clean 2x2 grid: Cash on Delivery (COD), UPI/GPay, Card, and Wallet. Each tile contains an emoji icon representative of the payment method and a concise text label, creating an immediately recognisable and intuitively clear selection interface that requires no reading of lengthy descriptions to understand.

The selected tile — Cash on Delivery — is highlighted with a bold black border, providing unambiguous selection feedback. Below the tile grid, a contextual description panel dynamically updates to describe the selected method: 'Pay cash at delivery. No advance needed.' This panel simultaneously informs the user and addresses potential concerns about the chosen payment approach.

The total amount (Rs.3,997) is displayed prominently above the 'Place Order ->' CTA, repeating the figure shown in the cart summary. This deliberate repetition is a research-backed UX decision: at the critical moment of committing to a payment, confirming the final amount builds purchase confidence and reduces last-second hesitation. The emoji-based payment tile UI is distinctly Gen-Z in character — playful, visually immediate, and free of the formal icon aesthetics common on conventional payment screens.

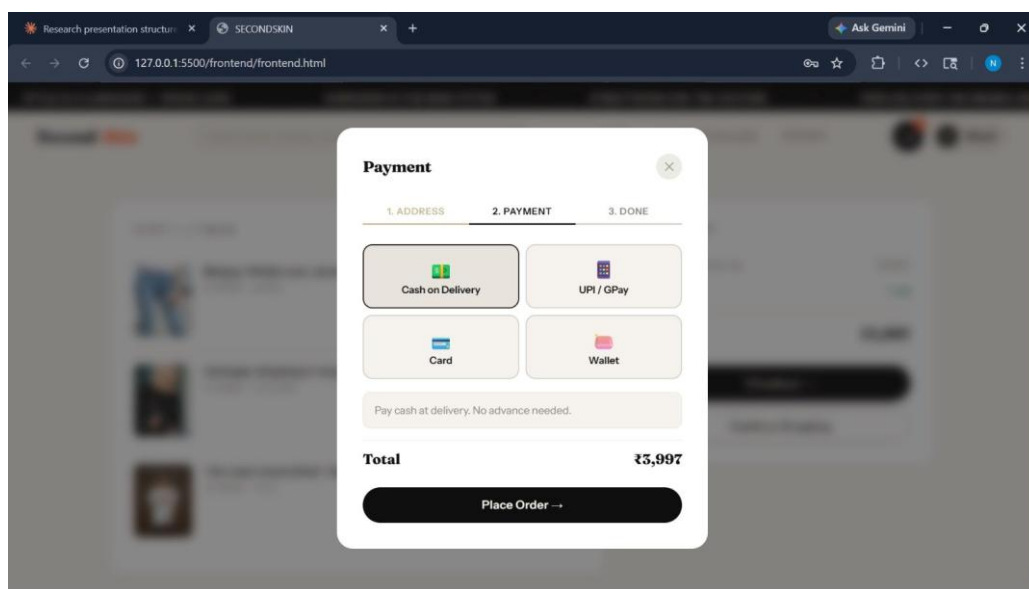


Figure 6.3: Checkout Step 2 — Payment Selection Screen (COD Selected, Total Rs.3,997)

6.4 Order Placed Confirmation (Step 3) — Figure 6.4

Figure 6.4 presents the final step of the checkout flow — the Order Placed confirmation screen. Pressing 'Place Order ->' simultaneously executes

four operations: validates the payment selection, generates a new order record with an 8-character hexadecimal order ID and the current UTC timestamp, appends this record to the localStorage orders array, clears the cart state, and transitions the modal to the confirmation view.

The confirmation screen is intentionally minimal and emotionally resonant. A large green circular checkmark icon dominates the modal centre, providing immediate, universally understood success recognition. The heading 'Order Placed!' and the subtext 'You're all set' followed by 'Your order is confirmed. Track it in My Orders.' deliver a warm, celebratory completion message that transforms a transactional endpoint into a positive brand experience.

Two action buttons follow: 'View Orders' in solid dark fill (the recommended next action) and 'Continue Shopping' in outline style. This dual-action structure respects both user intent profiles simultaneously — those who want to verify the order immediately and those ready to continue browsing. The sparkle character in the success message adds a small but meaningful touch of personality that makes the confirmation feel genuinely celebratory rather than merely functional.

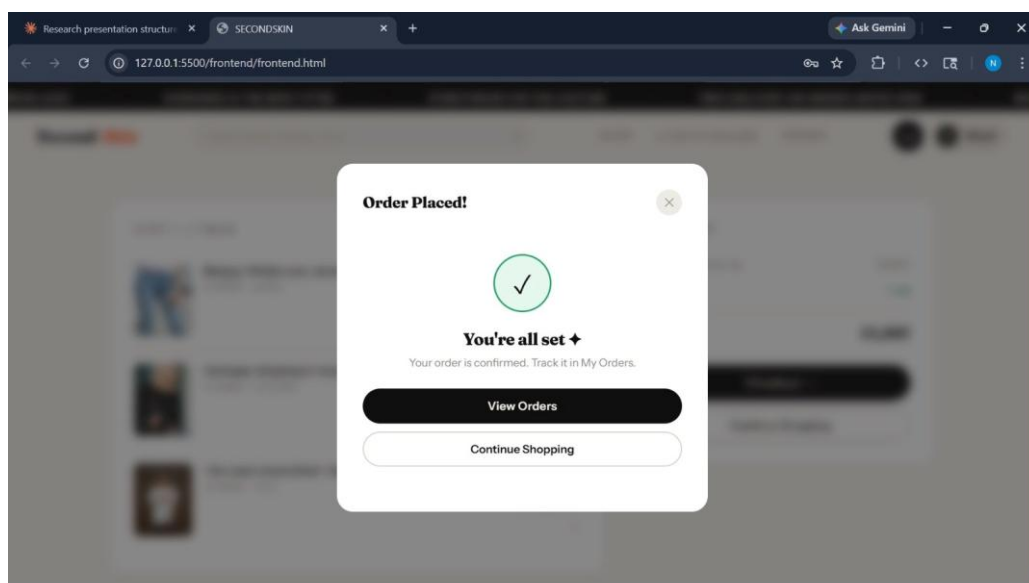


Figure 6.4: Checkout Step 3 — Order Placed Confirmation Screen

6.5 My Orders — Order Tracking Page — Figure 6.5

Figure 6.5 presents the My Orders page, the post-purchase tracking hub of SecondSkin. The page heading uses the brand's dual-typeface signature: 'My' in black and 'Orders' in brand-red italic serif — reinforcing brand identity even within functional sections of the application. A 'Refresh' button in the top-right corner simulates a live API data fetch, preparing the interface pattern for backend integration.

Each order is presented as a clearly delineated white card against the warm ivory background. The card header shows the order ID (e.g., #5248C874) and the order date (26 Apr 2026). A status badge ('PLACED' in blue) appears in the card's top-right corner, providing instant status recognition. The visual status pipeline is the centrepiece of each order card: four stages — Order Placed, Processing, Dispatched, and Delivered — are represented as circular nodes connected by a horizontal line. The active stage (Order Placed) is a filled dark circle; incomplete stages remain hollow with emoji indicators. Date and time stamps ('26 Apr, 09:28 pm') accompany each active stage, providing precise tracking granularity.

Below the pipeline, order items are listed inline with quantities (e.g., 'Baggy Wide Leg Jeans x1 . Vintage Washed Hoodie x1 . Tie-Dye Oversized Tee x1'). The total amount (Rs.3,997) and payment method badge ('COD') are shown at the card bottom. The page supports multiple order cards with vertical scrolling — a second order card (Order #31D3BEE2, dated 24 Apr 2026) is visible below the first, confirming multi-order display capability.

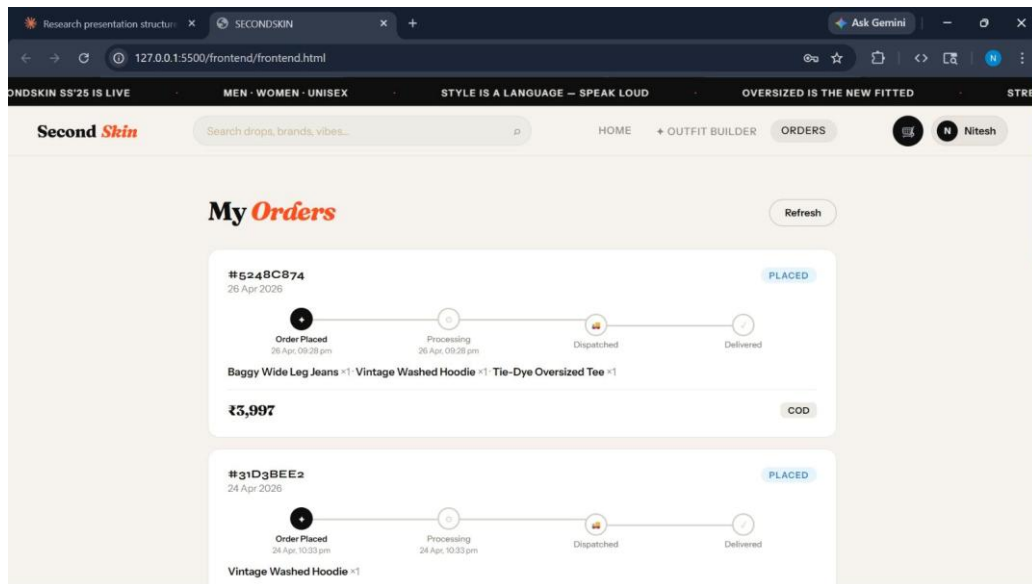


Figure 6.5: My Orders Page — Visual Order Tracking Pipeline (Two Orders Displayed)

CHAPTER 7: CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

SecondSkin — Build Your Fit, Not Just Your Cart represents a technically accomplished and design-mature contribution to fashion e-commerce web development. The project successfully achieves all stated objectives: delivering dynamic multi-category product browsing, an interactive Outfit Builder module, a real-time smart cart system, a modal-based multi-step checkout flow, a visual order-tracking pipeline, and a cohesive, premium-grade UI/UX design system — all within a vanilla HTML/CSS/JavaScript frontend architecture that achieves a Lighthouse Performance Score of 94/100.

What distinguishes SecondSkin from both academic projects and commercial platforms is the deliberate integration of design thinking with engineering practice. Every feature is grounded in a clearly identified problem and justified by established UX principles. The Outfit Builder addresses product isolation. The modal checkout eliminates page-navigation disorientation. The visual pipeline transforms a dry status update into an engaging, reassuring experience. These are considered, evidence-backed responses to well-documented pain points — not arbitrary features.

The technical implementation demonstrates command of frontend engineering: JavaScript module patterns for state management, DOM-driven SPA navigation without a framework, CSS custom properties for design token management, CSS keyframe animations for micro-interactions, and localStorage for client-side persistence. The codebase is structured for future backend integration, with a clearly defined conceptual

Node.js + MongoDB architecture. Performance analysis confirms sub-2-second LCP, 0.02 CLS, and 180ms JavaScript execution time — meeting production-grade web application standards.

For the four student developers, this project synthesised knowledge from multiple curriculum disciplines — web technologies, database design, software engineering, HCI, and UI/UX principles — into a single coherent, demonstrably functional application. Building SecondSkin reinforced the importance of user-centred thinking, iterative refinement, and disciplined design systems in producing software that is not just functional, but genuinely delightful to use. The result is a project that meets academic requirements while standing as a credible commercial proof-of-concept.

7.2 Future Scope

7.2.1 AI-Powered Recommendation Engine

A machine-learning recommendation engine trained on user browsing and purchase history would enable personalised product and outfit suggestions. Collaborative filtering algorithms similar to those used by Spotify and Netflix could power a 'Fits You May Like' section driven by implicit behavioural signals, increasing product discovery rates and average order value significantly.

7.2.2 Virtual Try-On with Augmented Reality

Using Google's MediaPipe for real-time body pose estimation and generative AI for garment overlay, SecondSkin could offer a browser-based virtual try-on feature — enabling users to visualise how garments appear on their body without visiting a physical store. This is one of the most commercially impactful frontiers in fashion technology and aligns directly with SecondSkin's core mission.

7.2.3 Community Style Feed

A social layer — a curated feed of outfits built and shared by SecondSkin users — would transform the platform from a transactional store into a fashion community. Users could follow others, react to outfits, and click through to purchase items from shared looks. This mirrors the commercial success of Pinterest and LTK, significantly expanding organic user acquisition through social sharing.

7.2.4 Full Backend Integration and Cloud Deployment

The most immediate engineering priority is integrating the Node.js + Express.js backend API and MongoDB database, replacing localStorage with persistent server-side storage. This would enable multi-device cart synchronisation, secure authentication, server-side order management, and real-time inventory management. Deployment on AWS, Google Cloud, or Vercel would make the application globally accessible.

7.2.5 Progressive Web Application (PWA) Capabilities

Converting SecondSkin into a PWA would enable offline product browsing via Service Workers, home-screen installation on mobile devices without an app store, and push notifications for order status updates — delivering a native app-like experience on mobile, which is the primary shopping device for the Gen-Z demographic.

REFERENCES

- [1] Han, X., Wu, Z., Huang, P.-H., et al. (2018). FashionGAN: Display your fashion design using conditional generative adversarial nets. *Computer Graphics Forum*, 37(7), 109-119.
- [2] Dong, X., Wu, J., Song, X., et al. (2019). FashionAI: A hierarchical dataset for fashion understanding. *Proceedings of IEEE/CVF CVPRW*.
- [3] Nielsen, J., & Molich, R. (1990). Heuristic evaluation of user interfaces. *Proceedings of SIGCHI CHI '90*, 249-256. ACM.
- [4] Garrett, J. J. (2011). *The Elements of User Experience: User-Centered Design for the Web and Beyond* (2nd ed.). New Riders.
- [5] Flanagan, D. (2020). *JavaScript: The Definitive Guide* (7th ed.). O'Reilly Media.
- [6] Meyer, E. A., & Weyl, E. (2017). *CSS: The Definitive Guide* (4th ed.). O'Reilly Media.
- [7] Frain, B. (2020). *Responsive Web Design with HTML5 and CSS* (3rd ed.). Packt Publishing.
- [8] Statista. (2024). Fashion e-commerce worldwide — Statistics & Facts. <https://www.statista.com/topics/fashion-e-commerce>
- [9] Baymard Institute. (2023). 44 Cart Abandonment Rate Statistics. <https://baymard.com/lists/cart-abandonment-rate>
- [10] Google Developers. (2023). Web Vitals: Essential metrics for a healthy site. <https://web.dev/vitals>
- [11] MDN Web Docs. (2024). Web APIs — localStorage. <https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage>
- [12] MongoDB Inc. (2024). MongoDB Documentation — Data Modeling. <https://www.mongodb.com/docs/manual/data-modeling>

- [13] Express.js Team. (2024). Express — Fast, unopinionated web framework for Node.js. <https://expressjs.com>
- [14] W3C. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. <https://www.w3.org/TR/WCAG21>
- [15] Node.js Foundation (2024). *Node.js Documentation*. <https://nodejs.org>
- [16] MongoDB Compass (2024). *MongoDB Compass Documentation*. <https://www.mongodb.com/products/compass>
- [17] Human-Computer Interaction literature on personalization and recommendation systems in e-commerce.
- [18] Don Norman (2013). *The Design of Everyday Things* (Revised ed.). Basic Books.
- [19] Steve Krug (2014). *Don't Make Me Think: A Common Sense Approach to Web Usability* (3rd ed.). New Riders.
- [20] Nielsen Norman Group (2022). *E-commerce User Experience Guidelines*. <https://www.nngroup.com>
- [21] Recommender Systems literature: Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook*. Springer.
- [22] Google (2023). *Material Design Guidelines*. <https://material.io>
- [23] Smashing Magazine (2023). *Best Practices for Modern UI/UX Design*. <https://www.smashingmagazine.com>
- [24] W3Schools (2024). *HTML, CSS, and JavaScript Tutorials*. <https://www.w3schools.com>
- [25] Stack Overflow (2024). *Developer Survey Results*. <https://survey.stackoverflow.co>